

Beyond Boolean Intersections: A Type-Safe Mechanism for Extracting Certified Shannon Bounds from D-Separated Traces

LUCIA YIZI ZHANG, Shanghai Lixin University of Accounting and Finance, China

We give a type-safe route from qualitative d-separation to certified Shannon-style leakage bounds. The route has four links: d-separation, conditional independence, conditional data processing, and a cut-set bound. This arXiv artifact mechanizes the graph-semantic front end in Lean 4; a separate anonymized artifact mechanizes the finite quantitative information-flow chain and is cited anonymously [1].

The graph-semantic core formalizes the two standard d-separation characterizations—trail blocking [8] and moralized ancestral-graph separation [5]—as a verified bisimulation between an operational semantics of active trails and a static analysis of moralized-graph reachability. Its three layers are a type-safe graph syntax whose pairwise-disjointness invariant ($\text{DisjointSets } X \ Y \ Z$) acts as an affine ownership constraint; a type-indexed intermediate representation (MAGWalk, StaticRoute, OpenTrace, ActiveRoute); and constructive witness-extraction pipelines in both directions.

The forward direction is a certified trace optimizer: it compiles any active trail through a Bayes-ball state machine into a compressed moral-graph walk, certifying soundness (moral-graph separation $\Rightarrow$ trail blocking) under the exact domain restriction that repairs equivalence. The reverse direction is a constructive decompiler of semantic flow: from moral-graph reachability we reconstruct an explicit active-trail witness, including a proved bad-collider normalization that strictly decreases a well-founded measure. Both directions compose into a closed equivalence on the pairwise-disjoint domain ($\text{dSeparated_iff_dSeparates}$); the graph core carries no proof debt. The local `InfoTheoryBridge.lean` file contains two intentional scaffold stubs that restate results proved in the anonymized companion artifact.

1 Introduction

The target is a certified route from a graph query to a quantitative information-flow statement:

$\text{d-separation} \Rightarrow \text{conditional independence} \Rightarrow \text{conditional DPI} \Rightarrow \text{cut-set Shannon bound}.$

This arXiv artifact contains the graph-semantic part of that route. A separate anonymized artifact [1] proves the semantic d-separation-to-CI bridge, conditional Markov instances, cut-set bounds, capacity certificates, and linear-chain DAG instances. The local bridge file in this repository is therefore a scaffold restating companion-proved results, not an in-package proof of the quantitative chain.

The graph front end still has a sharp technical problem. Consider the simplest possible directed acyclic graph: a chain of three nodes $0 \rightarrow 1 \rightarrow 2$. Let $X = \{0\}$, $Y = \{1\}$, and $Z = \{0\}$. Does Z d-separate X from Y ?

The two standard textbooks give different answers:

- **Trail blocking** [8] says: examine every undirected trail from X to Y ; if every internal triple is blocked by Z , the sets are d-separated. The one-edge trail $0 \rightarrow 1$ has no internal triple, so it is not blocked. Hence X and Y are *not* d-separated.
- **Moral graph separation** [5] says: take the ancestral subgraph of $X \cup Y \cup Z$, moralize it, delete Z , and check whether X and Y are disconnected. Node 0 is in Z , so it is deleted. The remaining graph has no path from 0 to 1, so X and Y *are* d-separated.

We mechanize this discrepancy and its repair, treating the two characterizations not as interchangeable definitions but as a *verified bisimulation* between two semantic layers:

- an **operational semantics** of active trails (the “execution” layer of information flow);

- a **static analysis** of moralized-graph reachability (the “compiled” layer).

The forward direction (soundness) is a *certified trace optimizer* that compiles an operational trace into static reachability evidence. The reverse direction (completeness) is a *constructive decompiler of semantic flow*: it reconstructs a concrete execution witness—an exploitability certificate for the information leak—from static reachability alone. The counterexample says exactly that the two languages are *not* bisimilar without a well-typed domain restriction.

1.1 Contributions

Our contributions are:

- (1) We formalize both d-separation characterizations in Lean 4 over finite DAGs with natural-number nodes, organized as a three-layer stack: type-safe graph syntax, certified intermediate representations, and constructive witness-extraction pipelines.
- (2) We construct and machine-check a concrete counterexample, proving that unrestricted equivalence is false and identifying the exact repair ($\text{DisjointSets } X \ Y \ Z$).
- (3) We mechanize the **forward bisimulation** (soundness): an explicit compilation pipeline $\text{Trail} \rightarrow \text{BayesBallPath} \rightarrow \text{MAGWalk} \rightarrow \text{Reachable}$, certified by node-survival proofs.
- (4) We mechanize the **reverse bisimulation** (witness-producing completeness): a decompilation pipeline $\text{Reachable} \rightarrow \text{StaticRoute} \rightarrow \text{OpenTrace} \rightarrow \text{ActiveRoute} \rightarrow \exists \text{ Trail}, \neg \text{isBlocked}$. Every stage is type-indexed and constructive, including a proved bad-collider normalization (`exists_split`, `ancestor_escape`, `route_improves_of_bad`) that strictly decreases a well-founded measure. The reverse pipeline carries no `sorry`.
- (5) We close the **full equivalence** (`dSeparated_iff_dSeparates`): on the pairwise-disjoint domain, moral-graph separation holds if and only if every trail is blocked, with both directions discharged by the optimizer and the decompiler.
- (6) We connect the graph core to an anonymized quantitative information-flow artifact [1], which proves the semantic bridge factorizes `dsep_implies_cond_indep`, cut-set theorem `abstract_cut_set_bound`, KKT capacity certificate `capacity_le_of_kkt`, and end-to-end DAG-routed instance `linear_chain_cut_set_bound_from_dag`. This arXiv artifact does not import that project; its local `InfoTheoryBridge.lean` file is an explicit scaffold restating the companion-proved bridge.

The graph results are available as a standalone Lean 4 package. The decompiler produces not a mere existence proof but a computational witness—a Σ -type containing the actual trail and the proof that it is unblocked.

2 A Three-Layer Architecture

Our formal development is organized in three layers that mirror the architecture of a verified compiler.

2.1 Layer 1: Type-Safe Graph Syntax

Nodes are natural numbers, edges are a finite set of ordered pairs ($\text{Finset } (\mathbb{N} \times \mathbb{N})$), and acyclicity is well-foundedness of the edge relation. The key safety invariant is $\text{DisjointSets } X \ Y \ Z$, which requires pairwise disjointness of the three node sets. In the language of substructural types, this is an *affine ownership constraint*: the conditioned variables Z are borrowed by the query and cannot alias the source X or sink Y . As planned future work for the POPL track—not part of this artifact—we target a first-order surface calculus with an explicit environment and no higher-order binders, so the metatheory would stay focused on queries rather than substitution-heavy closure machinery.

In that planned layer, surface graphs would carry a rank certificate for acyclicity, checked locally and then elaborated into intrinsic DAGs via `DAG.ofRank`.

Listing 1. DAG structure and safety predicate

```

structure DAG where
  nodes : Finset \N
  edges : Finset (\N \times \N)
  edges_subset : \forall {u v}, (u,v) \in edges -> u \in nodes \wedge v \in nodes
  acyclic : WellFounded fun u v => (u,v) \in edges

def DisjointSets (X Y Z : Finset \N) : Prop :=
  Disjoint X Y \wedge Disjoint X Z \wedge Disjoint Y Z

```

2.2 Layer 2: Certified Intermediate Representations

Between operational trails and graph reachability sits a family of IRs that preserve just enough structure for certified translation.

MAGWalk. A compressed walk language equivalent to moral-graph reachability. It has three constructors: reflexivity, a single directed or reversed edge (`MAGWalk.single`), and a “moral jump” over an active collider (`MAGWalk.jump`). The collider node is skipped, which is why compression is correct.

StaticRoute. A type-valued decompiler IR that keeps the witness hidden by graph reachability. Each step is either a forward edge, a backward edge, or a moral jump storing the common child:

Listing 2. Static route IR (excerpt)

```

inductive StaticStep (G : DAG) (X Y Z : Finset \N) : \N -> \N -> Type where
| directForward (hEdge : G.HasEdge u v) ...
| directBackward (hEdge : G.HasEdge v u) ...
| moralJump (hvw : G.HasEdge u w) (hvw : G.HasEdge v w)
  (hw : w \in G.ancestralSubgraphNodes (X \cup Y \cup Z)) ...

```

OpenTrace. The compiler target before converting to an active route. Each cons node stores its oriented DAG traversal *and* a local non-blocking proof, making *OpenTrace* a type-valued certificate that every junction is *Z*-open.

2.3 Layer 3: Witness-Extraction Pipelines

The two pipelines are summarized in Figure 1.

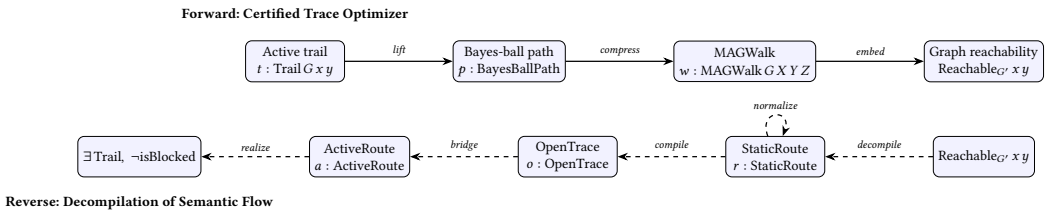


Fig. 1. The two witness-extraction pipelines. Solid arrows: forward (soundness). Dashed arrows: reverse (completeness). Every arrow, including the *normalize* step, is a total constructive function in Lean; the *normalize* step is a proved bad-collider minimization (Section 6).

3 Background: The Two Criteria

3.1 Trail Blocking (Operational Semantics)

An undirected *trail* from x to y in a DAG G is a finite walk that may follow edges forward or backward. A consecutive triple $a \rightarrow b \rightarrow c$ or $a \leftarrow b \rightarrow c$ is a *non-collider*; $a \rightarrow b \leftarrow c$ is a *collider*. A triple is *blocked* by Z if:

- it is a non-collider whose middle node is in Z ; or
- it is a collider whose middle node and all descendants are disjoint from Z .

A trail is *blocked* by Z if some internal triple is blocked. Then Z *d-separates* X from Y in the trail sense if every trail from X to Y is blocked by Z .

3.2 Moral Graph Separation (Static Analysis)

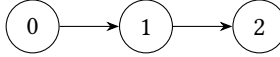
The *ancestral subgraph* of S is the subgraph induced by all ancestors of nodes in S . Its *moral graph* is obtained by (1) forgetting edge directions and (2) connecting all co-parents (pairs of nodes with a common child). The *d-separation graph* is the moral graph of the ancestral subgraph of $X \cup Y \cup Z$ with Z removed. Then Z d-separates X from Y in the graph sense if no node of X is connected to any node of Y in this graph.

3.3 Textbook Equivalence and Its Caveat

Standard references state that the two criteria are equivalent under the implicit assumption that X , Y , and Z are pairwise disjoint. In applications, however, the disjointness precondition is often omitted. Our counterexample (Section 4) shows that this omission is not benign.

4 The Endpoint Caveat: A Counterexample

Our counterexample lives on the simplest non-trivial DAG:



Let $X = \{0\}$, $Y = \{1\}$, $Z = \{0\}$.

THEOREM 4.1 (ENDPOINT-IN- Z COUNTEREXAMPLE). *There exists a DAG G and finite sets $X, Y, Z \subseteq \mathbb{N}$ such that G d-separates X from Y in the moral-graph sense, but Z does not d-separate X from Y in the trail-blocking sense.*

Listing 3. Mechanized counterexample

```

theorem dsep_complete_endpoint_in_Z_counterexample :
  \exists (G : DAG) (X Y Z : Finset \mathbb{N}),
    DAG.dSeparated G X Y Z \wedge \neg dSeparated G X Y Z := by
  refine <chain3, {0}, {1}, {0}, ?_, ?_>
  ...

```

Why the disagreement happens. Trail blocking only inspects *internal* triples. The one-edge trail $0 \rightarrow 1$ has no internal triple at all, so nothing can block it. By contrast, the moral-graph construction deletes Z unconditionally; since $0 \in Z$, the edge $(0, 1)$ disappears from the moralized graph, and 0 and 1 are disconnected.

The standard repair. If we require $X \cap Z = \emptyset$ and $Y \cap Z = \emptyset$, the start node of any trail is never deleted, and the endpoint caveat disappears.

5 Forward Bisimulation: The Certified Trace Optimizer

Under $\text{DisjointSets } X \ Y \ Z$, we mechanize the soundness direction:

$$\text{DAG.dSeparated}(G, X, Y, Z) \implies \text{dSeparates}(G, X, Y, Z).$$

The proof follows a three-stage compilation pipeline.

Stage 1: Trail $\rightarrow$ Bayes-ball path. If a trail t from $x \in X$ to $y \in Y$ is not blocked by Z , and $x \notin Z$ (guaranteed by $\text{Disjoint } X \ Z$), then we lift t to an explicit Bayes-ball state path. The construction is by induction on the trail, analyzing each directional triple to show that the Bayes-ball state machine can traverse it.

Listing 4. Trail to Bayes-ball path

```
def bayesBallPath_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z) :
  \Sigma final_dir, BayesBallPath G Z (u, outOf) (v, final_dir)
```

Stage 2: Certified membership. The Bayes-ball path contains intermediate states that must “survive” deletion of Z . We prove a certified version that guarantees every `RequiredState` node belongs to the d-separation graph nodes:

Listing 5. Certified Bayes-ball path

```
def bayesBallPathCert_of_active_trail_outOf
  (t : Trail G u v) (h_active : \neg t.isBlocked Z)
  (huZ : u \notin Z)
  (huA : u \in G.ancestralSubgraphNodes (X \cup Y \cup Z))
  (hvD : v \in G.dSeparationGraphNodes X Y Z) :
  \Sigma final_dir, {p : BayesBallPath ... //
    \forall q, BayesBallPath.RequiredState p q ->
      q.1 \in G.dSeparationGraphNodes X Y Z}
```

Stage 3: Compress to MAGWalk. A Bayes-ball path is a sequence of single-edge steps. The compress algorithm scans it with a two-step window: ordinary non-collider windows become a `MAGWalk.single`, while active-collider windows $(a, _) \rightarrow (b, \text{into}) \rightarrow (c, \text{outOf})$ become a single `MAGWalk.jump` over the collider b . The collider itself need not survive deletion of Z , which is why this compression is correct.

THEOREM 5.1 (SOUNDNESS). *For any DAG G and pairwise-disjoint finite sets $X, Y, Z \subseteq \mathbb{N}$, if G d-separates X from Y in the moral-graph sense, then every trail from X to Y is blocked by Z .*

Listing 6. Top-level soundness theorem

```
theorem dSeparated_of_dSeparated_disjoint
  (hXYZ : DisjointSets X Y Z)
  (hsep : DAG.dSeparated G X Y Z) : dSeparates G X Y Z
```

6 Reverse Bisimulation: Decompilation of Semantic Flow

The reverse direction is the decompiler: from $\neg\text{DAG.dSeparated}$ it constructively reconstructs an active-trail witness—an exploitability certificate showing that the conditioning set Z fails to block the flow. The pipeline is:

$$\begin{aligned} \text{Reachable} &\rightarrow \text{StaticRoute} \rightarrow \text{NormalizedStaticRoute} \\ &\rightarrow \text{OpenTrace} \rightarrow \text{ActiveRoute} \rightarrow \exists \text{Trail}, \neg\text{isBlocked}. \end{aligned}$$

6.1 From Reachability to StaticRoute

Graph reachability is a **Prop**-valued relation that hides the structure of each step. We introduce `StaticStep` and `StaticRoute` as type-valued evidence: a `StaticRoute` is an explicit list of directed edges, reversed edges, and moral jumps, each carrying the witness needed for later stages.

Listing 7. Reachability to static route

```
theorem nonemptyStaticRoute_of_dSeparationGraph_reachable
  (h : (G.dSeparationGraph X Y Z).Reachable u v) :
  Nonempty (StaticRoute G X Y Z u v)
```

6.2 Normalization: Eliminating Bad Colliders

A “bad collider” is a collider window whose middle node and all descendants are disjoint from Z . Such a collider is blocked, so it must be eliminated. Normalization may change endpoints: if a bad collider leaks ancestry into X or Y , we reroute the prefix or suffix to a different node in that set.

We package static routes as endpoint-carrying witnesses (`StaticRouteWitness`) and define a bad-collider count `routeBadCount`. The minimization is a `Nat.find` wrapper: if every nonzero-count witness can be strictly improved, a zero-count witness exists.

Listing 8. Bad-collider minimization wrapper

```
theorem normalized_route_exists_of_improves
  (hwit : \exists w : StaticRouteWitness G X Y Z, True)
  (himprove : \forall w, routeBadCount w \neq 0 ->
    \exists w', routeBadCount w' < routeBadCount w) :
  \exists w, routeBadCount w = 0
```

The improvement obligation is discharged by the core normalization theorem, which is fully proved (no **sorry**):

Listing 9. Strict improvement for non-normalized routes

```
theorem route_improves_of_bad
  (w : StaticRouteWitness G X Y Z) (hbad : routeBadCount w \neq 0) :
  \exists w' : StaticRouteWitness G X Y Z,
  routeBadCount w' < routeBadCount w
```

The proof is a short dispatcher over three established lemmas. `exists_split` extracts the first bad collider as a `Split` carrying a zero-bad prefix and a count bound. `ancestor_escape` shows the bad child’s leaked ancestry must reach a node in X or Y (its descendant cone misses Z , so it cannot dead-end). `bad_child_survives` and `escape_path_survives` prove that the rerouted escape chain stays inside the d-separation graph nodes. Reattaching the escape chain—backward into X or forward into Y —removes a moral jump of cost 1 and adds chains of cost 0; the strict decrease then follows by `countBadColliders_backwardEscape_append_suffix_lt` (resp. `countBadColliders_prefix_append_forwardEscape_lt`) and `omega`. As a deliberate design choice, the witness endpoints may change:

rerouting is allowed to land on a *different* node of X or Y , which is what makes the escape construction total.

6.3 From Zero-Bad StaticRoute to OpenTrace to ActiveRoute

A static route with zero bad colliders compiles directly to an OpenTrace, because every junction is locally open:

Listing 10. Zero-bad compiler

```
theorem openTrace_of_countBadColliders_zero
  (hzero : countBadColliders arrival route = 0) :
  Nonempty (\Sigma finalDir, OpenTrace G Z (x, arrival) (y, finalDir))
```

The OpenTrace is then converted to a BayesBallPath and wrapped as an ActiveRoute. Finally, ActiveRoute.to_activeTrail yields the existential witness:

Listing 11. Final decompiler theorem

```
theorem activeWitness_of_not_dSeparated
  (hnot : \neg DAG.dSeparated G X Y Z) : ActiveWitness G X Y Z
```

The result is a constructive Σ -type containing the actual nodes $x \in X$, $y \in Y$, the final direction, and the active route.

6.4 The Closed Equivalence

Composing the optimizer (Section 5) and the decompiler yields the full equivalence on the well-typed domain—the bisimulation statement: on pairwise-disjoint X, Y, Z , the operational and static languages agree exactly.

Listing 12. Full d-separation equivalence (no proof debt)

```
theorem dSeparated_iff_dSeparates
  (hXYZ : DisjointSets X Y Z) :
  DAG.dSeparated G X Y Z <=> dSeparates G X Y Z
```

REMARK 1 (VERIFICATION STATUS). *The full Lean development typechecks (lake build). The graph-semantic core—both directions of dSeparated_iff_dSeparates and the entire reverse synthesis stack—contains no **sorry**. The only **sorrys** in the package are two intentional scaffold stubs in InfoTheoryBridge.lean (Section 8), which are out of scope for the equivalence claims.*

7 The PL Dictionary: Reframing Causal Graphs

To situate this work for a POPL audience, we reframe traditional causal-graph vocabulary in programming-languages terms:

8 Related Work

Our mechanization sits at the intersection of causal inference, mechanized probabilistic semantics, and verified compilation. We briefly survey the most closely related lines of work.

Mechanized Causal Inference and D-Separation. Formalizing the graph-theoretic foundations of causal inference is an active frontier. Recently, Chancelier, De Lara, and Heymann (2024) [2] mechanized d-separation as a binary relation using Coq and MathComp/SSReflect, providing a relational-algebraic treatment of graphical separation. Parallel to our work, Anna Zhang (2025) [9]

Causal Inference	PL Framing
DAG	Dataflow AST / Dependency Graph Syntax
DisjointSets $X \ Y \ Z$	Affine ownership constraint (no aliasing of Z)
Active Trail	Operational trace of semantics
MAGWalk	Reachability abstract IR (compressed moral-graph walk)
Collider Jump (MAGWalk. jump)	Peephole optimization / semantic fold
Trail $\rightarrow$ BayesBallPath	Typestate tracking
BayesBallPath $\rightarrow$ MAGWalk	Certified trace optimizer
StaticRoute $\rightarrow$ OpenTrace $\rightarrow$ ActiveRoute	Semantic-flow decompiler / exploitability-certificate generator
Cut-Set / Mutual Information	Quantitative Information Flow (QIF) capacity bounds

Table 1. Reframing causal-graph concepts for a POPL audience.

formalized the semantics of conditional independence and d-separation in Coq, bridging structural properties with function-based definitions for experimental design.

Both developments adopt standard mathematical formulations without exposing the implicit textbook domain caveats. Our work contributes the first formalized counterexample and repair (the endpoint caveat) for the gap between syntactic trails and moralized node-deletion. We also shift from purely relational verification to a constructive bisimulation that generates executable IRs (e.g., OpenTrace), optimized in Lean 4 for computational witness synthesis.

Categorical and Topological D-Separation. Fritz and Klingler (2023) [3], extending work by Fritz and Liang (2023) [4], abstracted d-separation using string diagrams in Markov categories. Their framework bypasses moralization, reading conditional independence directly from topological connectedness in a generalized measure-theoretic space. Our approach is complementary: Markov categories provide a general topological theory, while our artifact targets the legacy operational graph definitions (moral graphs and trace trails) implemented in existing software. By defining DisjointSets as an affine ownership type, we show that classical d-separation behaves like a resource-restricted syntax within a broader categorical structure.

Verified Trace Bisimulation and Witness Extraction. The forward pipeline (soundness) translates unbounded trace semantics into a compressed step-relation (MAGWalk), analogous to the semantics-preserving translations in Leroy’s CompCert (2009) [6]. The reverse pipeline (completeness) deterministically unwinds reachability into a certified active trail, resembling Necula’s Proof-Carrying Code (1997) [7]: instead of returning a boolean assertion of an information leak, our decompiler synthesizes an explicit, machine-checkable exploitability certificate. This provides a graph-theoretic substrate for future verified static analyzers of quantitative information flow.

8.1 Quantitative Information-Flow Bridge

Certified Shannon bounds. Security analysis has moved from non-interference (“0 or 1”) to measuring leakage in bits. The verified bisimulation here supplies the graph-semantic front end for such bounds: a closed d-separation equivalence licenses replacing a conditional-independence premise by a checked graph property in a cut-set capacity argument. The full finite chain is

$$\text{d-separation} \Rightarrow \text{conditional independence} \Rightarrow \text{conditional DPI} \Rightarrow \text{cut-set bound}.$$

This arXiv artifact does not import the quantitative project. Instead, InfoTheoryBridge.lean states the first bridge as an explicit local scaffold: the conditional-independence predicate ConditionalIndependence and the bridge theorem dSeparation_implies_conditional_independence are stubbed alongside

placeholder structures `FinitePMF` and `MarkovCompatible`. This scaffold carries exactly two intentional `sorry`s, and it restates results proved in the separate anonymized artifact [1].

The companion artifact proves `factorizes_dsep_implies_cond_indep` under a stated `FactorizesOverDAG` hypothesis; the structural Verma–Pearl derivation of that hypothesis from a DAG factorization theorem remains a technical debt, not a gap in the semantic bridge theorem. It also proves `condMarkov_of_factorizes_dsep_fourVar`, `abstract_cut_set_bound`, `capacity_le_of_kkt`, and the end-to-end linear-chain result `linear_chain_cut_set_bound_from_dag`. The capacity premise `h_cap` in `abstract_cut_set_bound` is a theorem hypothesis, and the Gaussian-KL steps are carried as explicit external premises (`AssumesFanoBound`, `AssumesMissedCellBound`). The remaining open items are listed in Appendix A.

9 Conclusion

We formalized the two standard d-separation criteria in Lean 4 as a verified bisimulation between an operational semantics of active trails and a static analysis of moralized-graph reachability. We machine-checked a concrete counterexample to unrestricted equivalence, characterized the repair (pairwise disjointness as an affine ownership constraint), and discharged both directions: a certified trace optimizer for soundness and a constructive, witness-producing decompiler—with a proved bad-collider normalization—for completeness. The two compose into the closed theorem `dSeparated_iff_dSeparates`, and the graph-semantic core carries no proof debt. The local probabilistic bridge scaffold restates companion-proved results; what remains is the narrow technical debt listed below.

A Technical Debts

This appendix separates proved theorem chains from remaining work.

- (1) **Structural Verma–Pearl derivation.** The companion artifact proves the semantic bridge `factorizes_dsep_implies_cond_indep`: under `FactorizesOverDAG`, d-separation implies conditional independence. What remains open is the structural derivation of `FactorizesOverDAG` from a standard DAG factorization theorem.
- (2) **General capacity metatheorem.** The companion artifact proves concrete capacity certificates, including `capacity_le_of_kkt`, and uses them in cut-set bounds. A general Blahut–Arimoto/KKT concavity sufficiency theorem remains future work.
- (3) **Surface calculus and certified decision procedure.** The current artifact verifies the intrinsic DAG core. The planned first-order surface syntax, rank checker, elaboration soundness theorem, query conservativity theorem, and certified d-separation decision procedure are not yet mechanized.

Engineering note: the graph artifact and the quantitative artifact are two physically separate Lake projects with near-isomorphic DAG definitions and no shared imports yet. The two `sorry`s in this repository’s `InfoTheoryBridge.lean` file are a port placeholder for the proved semantic bridge in the cited anonymous artifact.

References

- [1] Anonymous. 2026. Certified Quantitative Information-Flow Bounds: A Finite Mechanized Cut-Set Pipeline. Companion artifact, under double-blind review; author and venue withheld for anonymity.
- [2] Jean-Philippe Chancelier, Michel De Lara, and Benjamin Heymann. 2024. Conditional Separation as a Binary Relation: A Coq Assisted Proof. Preprint. arXiv:2404.01777. Extended version available at hal-03315809.
- [3] Tobias Fritz and Andreas Klingler. 2023. The d-Separation Criterion in Categorical Probability. *Journal of Machine Learning Research (JMLR)* 24, 46 (2023), 1–49.
- [4] Tobias Fritz and Wendong Liang. 2023. Free gs-Monoidal Categories and Free Markov Categories. *Applied Categorical Structures* 31, 2 (2023), 21.

- [5] Steffen L. Lauritzen. 1996. *Graphical Models*. Oxford University Press, Oxford, UK.
- [6] Xavier Leroy. 2009. Formal verification of a realistic compiler. *Commun. ACM* 52, 7 (2009), 107–115.
- [7] George C. Necula. 1997. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, New York, NY, USA, 106–119.
- [8] Judea Pearl. 2009. *Causality* (2nd ed.). Cambridge University Press, Cambridge, UK.
- [9] Anna Zhang. 2025. *Formalizing Causal Models Through the Semantics of Conditional Independence*. Master's thesis. Massachusetts Institute of Technology. Supervised by Adam Chlipala.